

Workshop 3

Unit Tests Backend Documentation

Students:

Juan David Castaneda Cardenas

Laura Vanesa Alvarez Lafont

Nicolas Andres Bolanos Fernandez

Santiago Andres Fetecua Pulgarin

Sergio Nicolas Correa Escobar

Professor:

Carlos Andres Sierra Virguez

**Software Engineering II
School of Engineering
Universidad Nacional de Colombia**

Event Manager Microservice

Developed tests:

- **test_home:** checks that the microservice responds correctly on the main route (/).
- **test_create_event:** tests that an event can be created using the corresponding endpoint.
- **test_get_events_by_user:** verifies that events for a specific user can be retrieved.
- **test_delete_event:** checks that a created event can be successfully deleted.

```
===== test session starts =====
platform win32 -- Python 3.13.7, pytest-9.0.1, pluggy-1.6.0
rootdir: C:\Users\juanc\Documents\UNAL\Trabajos\Semestre 6\event_manager
plugins: anyio-4.11.0
collected 4 items

test\test_events.py .... [100%]

===== warnings summary =====
logic\schemas.py:12
  C:\Users\juanc\Documents\UNAL\Trabajos\Semestre 6\event_manager\logic\schemas.py:12: PydanticDeprecatedSince20: Support for class-based `config` is deprecated, use ConfigDict instead. Deprecated in Pydantic V2.0 to be removed in V3.0. See Pydantic V2 Migration Guide at https://errors.pydantic.dev/2.12/migration/
    class EventResponse(BaseModel):

test\test_events.py::test_create_event
test\test_events.py::test_delete_event
  C:\Users\juanc\Documents\UNAL\Trabajos\Semestre 6\event_manager\logic\crud.py:6: PydanticDeprecatedSince20: The `dict` method is deprecated; use `model_dump` instead. Deprecated in Pydantic V2.0 to be removed in V3.0. See Pydantic V2 Migration Guide at https://errors.pydantic.dev/2.12/migration/
    new_event = Event(**event.dict())

-- Docs: https://docs.pytest.org/en/stable/how-to/capture-warnings.html
===== 4 passed, 3 warnings in 0.19s =====
```

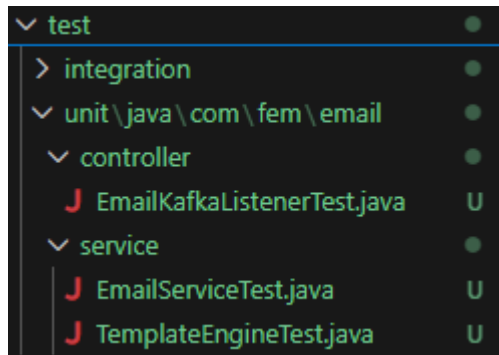
After modifications:

```
===== test session starts =====
platform win32 -- Python 3.13.7, pytest-9.0.1, pluggy-1.6.0
rootdir: C:\Users\juanc\Documents\UNAL\Trabajos\Semestre 6\event_manager
plugins: anyio-4.11.0
collected 4 items

test\test_events.py .... [100%]

===== 4 passed in 0.18s =====
```

Email Notification Microservice



Developed tests were separated in two components, the Pub/sub controller tests are presented below:

- **consume_ValidJson_CallsService:** Verifies that valid JSON is deserialized and the EmailService is called once.
- **consume_InvalidJson_ThrowsExceptionNotCrash:** Ensures invalid JSON does not crash the listener and the service is not invoked.
- **consume_ServiceThrowsException_ListenerContinues:** Checks that the listener handles EmailService exceptions without stopping and still calls the service once.

On the other hand, the email service tests:

- **sendEmail_Success:** Validates the successful email flow including template rendering, SMTP sending, and correct EmailResponse.
- **sendEmail_TemplateError_ReturnsErrorResponse:** Ensures template rendering errors produce a failure EmailResponse with the appropriate message.
- **sendEmail_SmtpFailure_ReturnsErrorResponse:** Verifies that SMTP failures result in a proper error response without throwing exceptions.
- **sendEmail_UnexpectedException_ReturnsErrorResponse:** Checks that unexpected internal exceptions are handled gracefully and returned as generic errors.

The last is specifically for the template engine subservice:

- **render_UsesCacheOnSecondCall:** Ensures the template is loaded only once and subsequent calls use the cache.

```
PS C:\Users\secoe\Documents\Projects\FastEventManager\src\java-backend\email> ./gradlew unitTest --warning-mode all
Java HotSpot(TM) 64-Bit Server VM warning: Sharing is only supported for boot loader classes because bootstrap classpath has been appended

BUILD SUCCESSFUL in 6s
4 actionable tasks: 1 executed, 3 up-to-date
PS C:\Users\secoe\Documents\Projects\FastEventManager\src\java-backend\email>
```

Test Summary

8

tests

0

failures

0

ignored

1.838s

duration

100%

successful

Packages		Classes			
Package	Tests	Failures	Ignored	Duration	Success rate
com.fem.email.controller	3	0	0	1.413s	100%
com.fem.email.service	5	0	0	0.425s	100%

Authentication Microservice

Test Summary

8	0	0	4.705s
tests	failures	ignored	duration

100%
successful

Packages	Classes				
Package	Tests	Failures	Ignored	Duration	Success rate
com.fem.authentication.service	5	0	0	1.361s	100%
com.fem.authentication.util	3	0	0	3.344s	100%

Developed tests were separated into two components, the Service tests are presented below:

- **testRegisterSuccess:** Verifies that the data sent to the database is the same as the sent from the new user.
- **testPasswordHashing:** Ensures hashing of password before saving it into the database.
- **testRegisterEmailExists:** Checks the error thrown if the email sent from the new user already existed in the database.
- **testLoginWithUsernameStrategy:** Validates the use of the correct logging strategy if the identifier sent was the username of the user.
- **testLoginWithEmailStrategy:** Validates the use of the correct logging strategy if the identifier sent was the email of the user.

On the other hand, the authentication util tests:

- **testTokenExpires:** Validates the expiration time of a generated JWT.
- **testErrorForInvalidToken:** Ensures verification of valid tokens, discarding and throwing errors in case of modified/invalid JWT.
- **testGenerateAndVerifyToken:** Checks that the generated JWT was valid and returned the email of the user.